06_tuned_models

September 8, 2026

1 06 — Tuned Models dengan SMOTE-ENN (GridSearchCV + StratifiedKFold)

Notebook ini melatih XGBoost & LightGBM pada **data training yang sudah di-resample SMOTE-ENN** (`X_train_resampled/y_train_resampled`), dengan hyperparameter tuning menggunakan **GridSearchCV + StratifiedKFold**.

Kunci anti-data-leakage:

- Hyperparameter tuning dilakukan **hanya di dalam training set** (cross-validation di dalam `X_train_resampled`), **tidak pernah** melihat test set.
- Evaluasi final menggunakan `X_test/y_test` **asli** (sama persis dengan baseline, belum di-resample).

Grid yang di-tuning:

- **XGBoost**: `n_estimators`, `max_depth`, `learning_rate`, `gamma`.
- **LightGBM**: `n_estimators`, `max_depth`, `learning_rate`, `num_leaves`.

Scoring CV = **F1 pada kelas positif (`Stunted=1`)**, konsisten dengan fokus metodologi (recall/F1 pada kelas minoritas lebih penting daripada accuracy).

[1]:
```
import os
import json
import pandas as pd
import numpy as np
from joblib import load

from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
)
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
```

```
RANDOM_STATE = 42
DATA_DIR = "../data/processed"
METRICS_DIR = "../results/metrics"
LABELS = ["Not Stunted", "Stunted"]
POS_LABEL = 1
```

[2]:
```
X_train_resampled = load(os.path.join(DATA_DIR, "X_train_resampled.joblib"))
y_train_resampled = load(os.path.join(DATA_DIR, "y_train_resampled.joblib"))
X_test = load(os.path.join(DATA_DIR, "X_test.joblib"))
y_test = load(os.path.join(DATA_DIR, "y_test.joblib"))

# Pastikan test set adalah test ASLI (belum di-resample), sama dengan baseline
assert X_test.shape[0] == 8013, "Test set tidak sama dengan baseline!"

print("X_train_resampled", X_train_resampled.shape, "| y_train_resampled",␣
 ↪y_train_resampled.shape)
print("X_test (asli)", X_test.shape, "| y_test (asli)", y_test.shape)
print("Distribusi y_train_resampled:", np.bincount(np.
 ↪asarray(y_train_resampled).astype(int)).tolist())
print("Distribusi y_test:", np.bincount(np.asarray(y_test).astype(int)).
 ↪tolist())
```

```
X_train_resampled (34756, 4) | y_train_resampled (34756,)
X_test (asli) (8013, 4) | y_test (asli) (8013,)
Distribusi y_train_resampled: [17218, 17538]
Distribusi y_test: [4773, 3240]
```

1.1 1. Setup Grid & Cross-Validation

`StratifiedKFold` (5 fold, shuffle, random_state=42) menjaga proporsi kelas di tiap fold selama tuning imbalanced-data terkait.

> **Catatan metodologi (batasan):** SMOTE-ENN diterapkan **sekali** pada training set di notebook 05, lalu GridSearchCV dijalankan di atas data yang sudah resample. Artinya sampel sintetis dapat terpecah antar fold CV, sehingga skor F1 CV cenderung sedikit optimistik. `X_test`/`y_test` tetap murni (dicek `assert`), jadi metrik final tetap valid; yang berisiko hanyalah ketepatan pemilihan hyperparameter. Ini dicatat sebagai batasan (bukan leakage).

[3]:
```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

xgb_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.05, 0.1, 0.2],
    "gamma": [0, 0.1, 0.3],
}
```

```
lgbm_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.05, 0.1, 0.2],
    "num_leaves": [15, 31, 63],
}
```

1.2 2. Tuning XGBoost

[4]:
```
xgb_base = XGBClassifier(
    random_state=RANDOM_STATE, eval_metric="logloss", verbosity=0
)
xgb_gscv = GridSearchCV(
    xgb_base, xgb_grid, scoring="f1", cv=skf, n_jobs=1, verbose=0
)
xgb_gscv.fit(X_train_resampled, y_train_resampled)

print("Best params XGBoost:", xgb_gscv.best_params_)
print(f"Best CV F1 (mean): {xgb_gscv.best_score_:.4f}")
```

```
Best params XGBoost: {'gamma': 0, 'learning_rate': 0.2, 'max_depth': 7,
'n_estimators': 300}
Best CV F1 (mean): 0.9965
```

1.3 3. Tuning LightGBM

[5]:
```
lgbm_base = LGBMClassifier(
    random_state=RANDOM_STATE, verbosity=-1
)
lgbm_gscv = GridSearchCV(
    lgbm_base, lgbm_grid, scoring="f1", cv=skf, n_jobs=1, verbose=0
)
lgbm_gscv.fit(X_train_resampled, y_train_resampled)

print("Best params LightGBM:", lgbm_gscv.best_params_)
print(f"Best CV F1 (mean): {lgbm_gscv.best_score_:.4f}")
```

```
Best params LightGBM: {'learning_rate': 0.1, 'max_depth': 7, 'n_estimators':
300, 'num_leaves': 63}
Best CV F1 (mean): 0.9969
```

1.4 4. Evaluasi Model Final pada Test Set (sama dengan baseline)

[6]:
```
final_models = {
    "XGBoost": xgb_gscv.best_estimator_,
    "LightGBM": lgbm_gscv.best_estimator_,
}
```

```python
def evaluate_model(name, model):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]
    tn, fp, fn, tp = confusion_matrix(y_test, y_pred, labels=[0, 1]).ravel()
    metrics = {
        "model": name,
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, pos_label=POS_LABEL),
        "recall": recall_score(y_test, y_pred, pos_label=POS_LABEL),
        "f1_score": f1_score(y_test, y_pred, pos_label=POS_LABEL),
        "roc_auc": roc_auc_score(y_test, y_proba),
        "tn": tn,
        "fp": fp,
        "fn": fn,
        "tp": tp,
    }
    return metrics, y_pred, y_proba


rows = []
for name, model in final_models.items():
    metrics, y_pred, _ = evaluate_model(name, model)
    rows.append(metrics)

    print(f"===== {name} (tuned, SMOTE-ENN) =====")
    print(classification_report(y_test, y_pred, target_names=LABELS, digits=4))
    cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
    print(pd.DataFrame(cm, index=[f"Actual {l}" for l in LABELS],
                       columns=[f"Pred {l}" for l in LABELS]))
    print()
```

```
===== XGBoost (tuned, SMOTE-ENN) =====
              precision    recall  f1-score   support

 Not Stunted     0.9629    0.9633    0.9631      4773
     Stunted     0.9460    0.9454    0.9457      3240

    accuracy                         0.9561      8013
   macro avg     0.9544    0.9544    0.9544      8013
weighted avg     0.9561    0.9561    0.9561      8013

                    Pred Not Stunted  Pred Stunted
Actual Not Stunted              4598           175
Actual Stunted                   177          3063

===== LightGBM (tuned, SMOTE-ENN) =====
```

```
              precision    recall  f1-score   support

 Not Stunted     0.9654    0.9600    0.9627      4773
     Stunted     0.9415    0.9494    0.9454      3240

    accuracy                         0.9557      8013
   macro avg     0.9535    0.9547    0.9541      8013
weighted avg     0.9558    0.9557    0.9557      8013

                     Pred Not Stunted  Pred Stunted
Actual Not Stunted               4582           191
Actual Stunted                    164          3076
```

1.5 5. Simpan Hasil

[7]:
```
results_df = pd.DataFrame(rows)
results_df
```

[7]:
```
      model  accuracy  precision    recall  f1_score   roc_auc    tn   fp  \
0   XGBoost  0.956071   0.945954  0.945370  0.945662  0.990805  4598  175
1  LightGBM  0.955697   0.941537  0.949383  0.945443  0.990885  4582  191

    fn    tp
0  177  3063
1  164  3076
```

[8]:
```
os.makedirs(METRICS_DIR, exist_ok=True)

results_df.to_csv(os.path.join(METRICS_DIR, "smoteenn_results.csv"),␣
 ↪index=False)
print("Tersimpan:", os.path.join(METRICS_DIR, "smoteenn_results.csv"))

best_params = {
    "XGBoost": xgb_gscv.best_params_,
    "LightGBM": lgbm_gscv.best_params_,
}
with open(os.path.join(METRICS_DIR, "best_params.json"), "w") as f:
    json.dump(best_params, f, indent=2)
print("Tersimpan:", os.path.join(METRICS_DIR, "best_params.json"))
```

```
Tersimpan: ../results/metrics\smoteenn_results.csv
Tersimpan: ../results/metrics\best_params.json
```